

Student Name: Ilyas Zazai

Student ID: 041173490

Course Code: CST8921 – Cloud Industry Trends

Lab Number: Lab 6

Lab Title: Code-First Static Web Delivery on Azure: SDK Provisioning, a Serverless API, and Edge Distribution

Submission Date: 6/21/2026

1. Introduction

In this lab, I built and deployed a static website on Azure using a code-first approach. The main goal was to avoid manual portal steps and use scripts to create, configure, deploy, and clean up Azure resources.

The website was hosted on Azure Storage Static Website. I also added an Azure Function API for a visit counter. After that, I used Azure Front Door to serve the website through an edge endpoint.

This lab helped me understand how cloud engineers deploy websites in a more repeatable way. Instead of clicking everything manually, I used Python SDK scripts, Azure CLI commands, and Azure Functions Core Tools.

In this lab, I used the eastus region because canadacentral was not available in my Azure lab subscription.

2. Architecture Flow

The architecture of this lab had three main parts.

First, Azure Storage hosted the static website files. These files were index.html, config.js, and 404.html.

Second, Azure Function worked as the backend API. The static website called this API to get the visit count.

Third, Azure Front Door was placed in front of the storage website. This gave the website an edge endpoint and allowed caching and cache purge.

The final flow was:

Browser



Azure Front Door



Azure Storage Static Website

↓

index.html and config.js

↓

JavaScript fetch call

↓

Azure Function API

↓

Azure Table Storage

↓

Return visit count to browser

The static part of the website came from Azure Storage. The dynamic visit counter came from Azure Function. The counter value was stored in Azure Table Storage.

This is similar to the Jamstack pattern because the frontend is static, but the dynamic data comes from a serverless API.

3. Lab Walkthrough with Screenshots

Screenshot-01-provision-output.png

```
import os
from azure.identity import DefaultAzureCredential ...

cred = DefaultAzureCredential()

resource_client = ResourceManagementClient(cred, SUBSCRIPTION_ID)
storage_client = StorageManagementClient(cred, SUBSCRIPTION_ID)

# Create or update resource group
resource_client.resource_groups.create_or_update(
    RG,
    {"location": LOCATION}
)

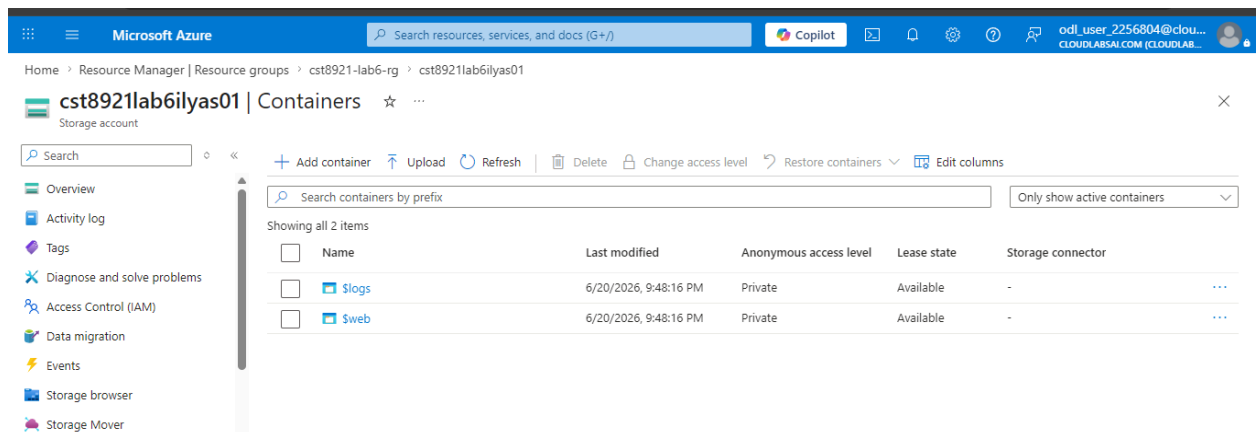
EOF print("Primary web endpoint will be available after static website hosting is enabled.")
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ python provision.py
Resource group 'cst8921-lab6-rg' ready in eastus.
Creating storage account 'cst8921lab6ilyas01'...
Storage account created successfully.
Account name: cst8921lab6ilyas01
Primary web endpoint: https://cst8921lab6ilyas01.z13.web.core.windows.net/
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$
```

Screenshot-02-static-website-enabled.png

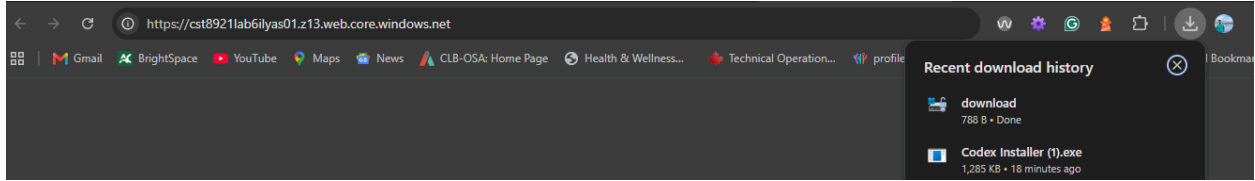
```
service = BlobServiceClient(account_url, credential=cred)

EOF print("Static website properties from Azure:")
ng parameter name.ually applied.t.
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ python enable_static_website.py
Static website hosting enabled successfully.
Static website properties from Azure:
{'enabled': True, 'index_document': 'index.html', 'error_document404_path': '404.html', 'default_index_document_path': None}
```

Screenshot-03-storage-web-endpoint.png



Screenshot-04-content-type-problem.png



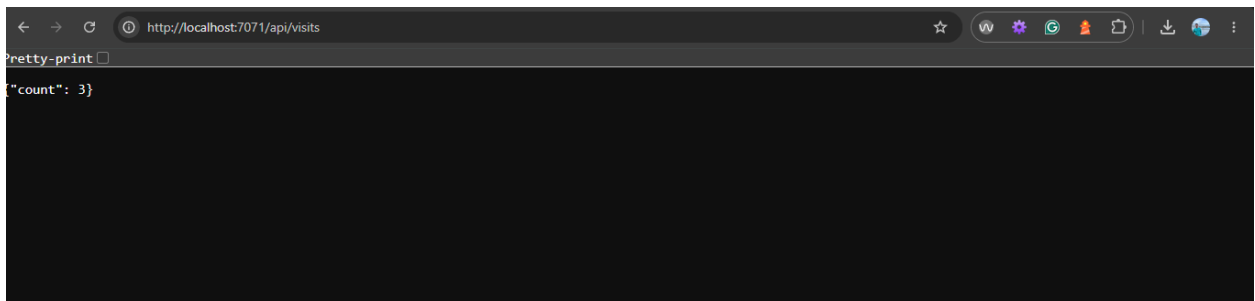
Screenshot-05-content-type-fixed.png



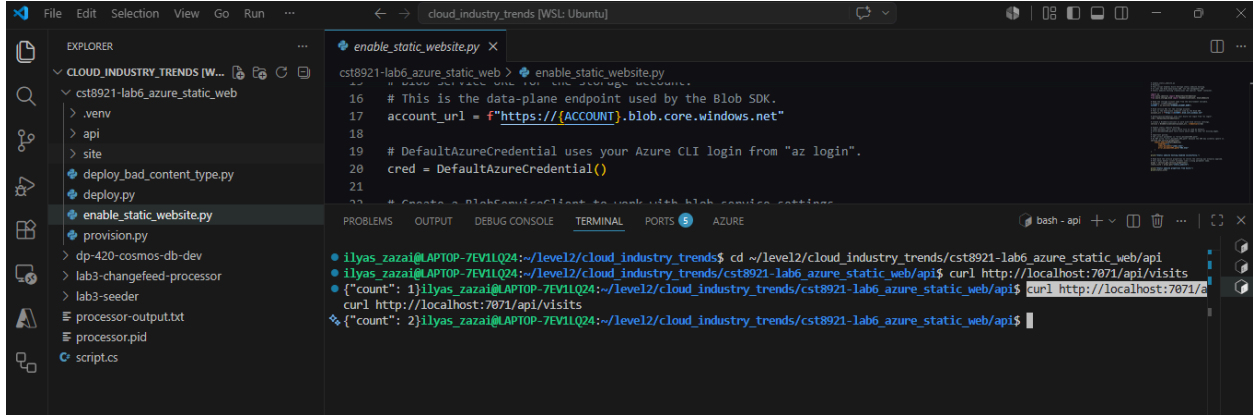
Hello from a code-deployed static site

Visits to this page: API unavailable

Screenshot-06-function-running-locally.png



Screenshot-07-api-counter-curl.png



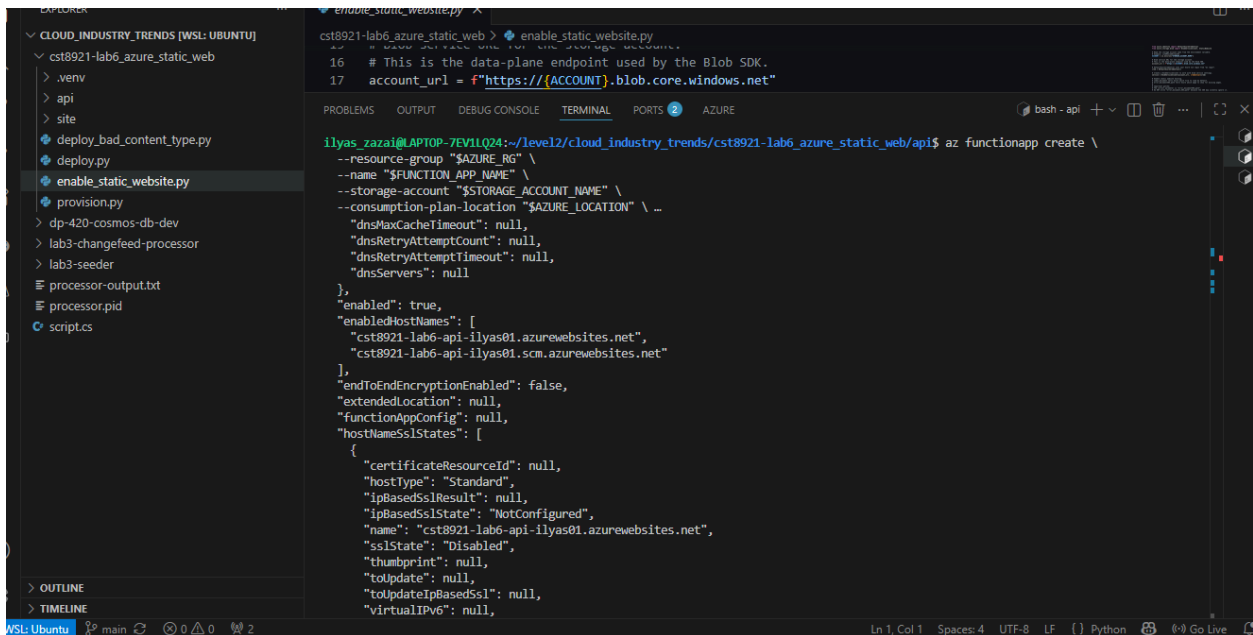
The screenshot shows a VS Code editor window with the file explorer on the left. The file explorer shows a project structure for 'cloud_industry_trends' with a sub-project 'cst8921-lab6_azure_static_web'. The file 'enable_static_website.py' is selected. The editor shows the following code:

```
15 # Enable static site for the storage account.
16 # This is the data-plane endpoint used by the Blob SDK.
17 account_url = f"https://{ACCOUNT}.blob.core.windows.net"
18
19 # DefaultAzureCredential uses your Azure CLI login from "az login".
20 cred = DefaultAzureCredential()
21
22 # Create a BlobServiceClient to work with blob service settings.
```

The terminal window at the bottom shows the following commands and output:

```
bash - api
ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends$ cd ~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api
ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api$ curl http://localhost:7071/api/visits
{"count": 1}
ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api$ curl http://localhost:7071/api/visits
{"count": 2}
```

Screenshot-08-function-app-created.png



The screenshot shows a VS Code editor window with the file explorer on the left. The file explorer shows a project structure for 'cloud_industry_trends' with a sub-project 'cst8921-lab6_azure_static_web'. The file 'enable_static_website.py' is selected. The editor shows the following code:

```
15 # Enable static site for the storage account.
16 # This is the data-plane endpoint used by the Blob SDK.
17 account_url = f"https://{ACCOUNT}.blob.core.windows.net"
```

The terminal window at the bottom shows the following command and output:

```
bash - api
ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api$ az functionapp create \
--resource-group "AZURE_RG" \
--name "$FUNCTION_APP_NAME" \
--storage-account "$STORAGE_ACCOUNT_NAME" \
--consumption-plan-location "$AZURE_LOCATION" \
--dns-max-cache-timeout null,
--dns-retry-attempt-count null,
--dns-retry-attempt-timeout null,
--dns-servers null,
--enabled true,
--enabled-host-names [
  "cst8921-lab6-api-ilyas01.azurewebsites.net",
  "cst8921-lab6-api-ilyas01.scm.azurewebsites.net"
],
--end-to-end-encryption-enabled false,
--extended-location null,
--function-app-config null,
--host-name-ssl-states [
  {
    "certificate-resource-id": null,
    "host-type": "Standard",
    "ip-based-ssl-result": null,
    "ip-based-ssl-state": "NotConfigured",
    "name": "cst8921-lab6-api-ilyas01.azurewebsites.net",
    "ssl-state": "Disabled",
    "thumbprint": null,
    "to-update": null,
    "to-update-ip-based-ssl": null,
    "virtual-ip-v6": null,
  }
]
```

Screenshot-09-function-published.png

```
ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api$ func azure functionapp publish "$FUNCTION"
N_APP_NAME"
Directory table size 8775 bytes (8.57 Kbytes)
  34.55% of uncompressed directory table size (25396 bytes)
Number of duplicate files found 45
Number of inodes 960
Number of files 837
Number of fragments 68
Number of symbolic links 0
Number of device nodes 0
Number of fifo nodes 0
Number of socket nodes 0
Number of directories 123
Number of ids (unique uids + gids) 1
Number of uids 1
  root (0)
Number of gids 1
  root (0)
Creating placeholder blob for linux consumption function app...
SCM_RUN_FROM_PACKAGE placeholder blob scm-latest-cst8921-lab6-api-ilyas01.zip located
Uploading built content /home/site/artifacts/functionappartifact.squashfs for linux consumption function app...
Resetting all workers for cst8921-lab6-api-ilyas01.azurewebsites.net
Deployment successful. deployer = Push-Deployer deploymentPath = Functions App ZipDeploy. Extract zip. Remote build.
Remote build succeeded!
[2026-06-21T03:15:40.983Z] Syncing triggers...
Functions in cst8921-lab6-api-ilyas01:
  visits - [httpTrigger]
    Invoke url: https://cst8921-lab6-api-ilyas01.azurewebsites.net/api/visits

ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web/api$
```

Screenshot-10-website-working-counter.png



Screenshot-11-front-door-endpoint.png



Screenshot-12-front-door-cache-purge.png

```
Azure Front Door cache purge completed successfully for /*
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ az afd endpoint purge -g "$AZURE_RESOURCE_GROUP" --profile-name cst8921-afd --endpoint-name lab6site --content-paths '/*'
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ echo "Azure Front Door cache purge completed successfully for /*"
Azure Front Door cache purge completed successfully for /*
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$
```

Screenshot-13-all-resources

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and user information. The main content area displays the 'cst8921-lab6-rg' resource group. The 'Resources' tab is selected, showing a list of resources. The list includes:

Name	Type	Location
Application Insights Smart Detection	Action group	Global
cst8921-afd	Front Door	Global
cst8921-lab6-api-ilyas01	Application Insights	East US
cst8921-lab6-api-ilyas01	Function App	East US
cst8921lab6ilyas01	Storage account	East US
EastUSLinuxDynamicPlan	App Service plan	East US
Failure Anomalies - cst8921-lab6-api-ilyas01	Smart detector alert rule	Global

Screenshot-14-all-resource-are-deleted

```
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ cat > cleanup.py <<'EOF'
# cleanup.py
# Purpose:
# Delete the whole Azure resource group created for this lab.
# ...
#
# The script is idempotent:
# If the resource group does not exist, it does not fail.

import os

from azure.identity import DefaultAzureCredential
from azure.mgmt.resource import ResourceManagementClient

# Read subscription ID and resource group name from environment variables.
SUBSCRIPTION_ID = os.environ["AZURE_SUBSCRIPTION_ID"]
RG = os.environ["AZURE_RG"]

# Use current Azure CLI login identity.
cred = DefaultAzureCredential()

# Management-plane client for Azure Resource Groups.
EOF print(f"Resource group '{RG}' does not exist. Nothing to delete.").
(.venv) ilyas_zazai@LAPTOP-7EV1LQ24:~/level2/cloud_industry_trends/cst8921-lab6_azure_static_web$ python cleanup.py
Deleting resource group 'cst8921-lab6-rg'...

Resource group deleted successfully.
```


4. Finding and Analysis

In this lab, I learned how to deploy a static website on Azure using a code-first method instead of manual portal steps. I used Python SDK scripts to create the resource group, create the storage account, enable static website hosting, upload files, and clean up resources.

One important finding was the difference between the control plane and the data plane. Creating the resource group and storage account was a control-plane task. Enabling static website hosting and uploading files to the \$web container were data-plane tasks. This is why I needed a Storage Blob Data role.

I also learned that uploading files is not enough. The files need correct metadata. When I uploaded index.html without the correct content type, the browser downloaded it instead of showing it as a web page. I fixed this by using ContentSettings in deploy.py.

The Azure Function made the static website dynamic. The website itself did not store the visit count. The browser called the Azure Function API, and the Function updated the counter in Azure Table Storage.

Azure Front Door added an edge endpoint in front of the storage website. It can improve delivery and caching, but old content may stay cached. For that reason, I used the Front Door purge command.

Overall, this lab showed me that real cloud deployment needs more than just creating resources. It also needs permissions, correct file metadata, API integration, caching, and cleanup.

5. Analysis Questions

Question 1: Three failure modes of the manual portal/deploy extension approach

The first problem is manual mistakes. If I use the Azure Portal, I can forget a setting or choose the wrong option.

The second problem is wrong file metadata. In my lab, when index.html was uploaded without the correct content type, the browser downloaded the file instead of rendering it.

The third problem is poor repeatability. If I do everything manually, it is hard to repeat the same setup again. With Python scripts, I can run the same steps again and get the same result.

Question 2: Why is static website hosting data-plane, but storage account creation is control-plane

Storage account creation is a control plane because it creates the Azure resource itself.

Static website hosting is data-plane because it changes Blob service properties and works with the \$web container inside the storage account.

Azure separates these permissions for security. A user may be allowed to create resources but not allowed to access or change the data inside them. That is why I needed the Storage Blob Data role for the static website and upload steps.

Question 3: Cache-Control issue and how to prevent a broken page

A broken page can happen if index.html is updated but the browser or Front Door still uses an old cached JavaScript file.

For example, if I change config.js to point to the new Azure Function API, but the browser keeps the old cached config.js, the website may still call the wrong API.

To prevent this, I can use versioned file names or cache-busting, such as:

config.js?v=2

In real projects, I can use asset hashing, like:

app.abc123.js

This makes the browser load the correct new asset.

Question 4: What I gained and what new problems/costs I added by using Azure Function

I gained dynamic behavior. The static website could show a real visit counter instead of only static text.

I also separated the frontend and backend. Azure Storage hosted the static files, and Azure Function handled the API logic.

But I also added new possible problems. The Function API can fail, Table Storage can fail, CORS can be wrong, or the API URL in config.js can be wrong.

I also added small extra costs because Azure Function, Table Storage operations, and Front Door can create charges.

Question 5: When Front Door caching can hurt correctness and how to fix it

Front Door caching can hurt correctness when users receive old content after a new deployment.

For example, if I update config.js with a new API URL but Front Door still serves the old cached file, the website may call the old API.

To fix this without disabling caching completely, I can purge the Front Door cache after deployment. I can also use no-cache for HTML and versioned file names for JavaScript and other assets.

5. Conclusion

In this lab, I built a static website delivery system on Azure using a code-first approach.

I created the resource group and storage account with Python SDK. I enabled static website hosting with the Blob SDK. I uploaded website files with correct content types and cache-control headers.

I also added an Azure Function API for a dynamic visit counter. The static website called this API from the browser, and the counter value was stored in Azure Table Storage.

Finally, I placed Azure Front Door in front of the storage website and demonstrated cache purge. At the end, I deleted the resources using a cleanup script.

The main thing I learned is that real cloud deployment is not only about creating services. It is also about identity, permissions, file metadata, caching, API integration, edge delivery, and cl